

NOM Prénom :
Nunéro Étudiant :

Université Paris Cité

Année 2022–2023

Programmation Réseaux - Partiel - Partie II

Lorsque vous aurez à modifier du code, vous pourrez juste écrire les lignes à modifier en précisant leur numéro ou les lignes à ajouter en précisant entre quelles lignes elles s'insèrent.

1. Soit le code suivant :

```
1 int sock = socket(PF_INET6, SOCK_STREAM, 0);
2 if(sock == -1)
3     exit(1);
4
5 struct sockaddr_in6 adrsock;
6 adrsock.sin6_family = AF_INET6;
7 adrsock.sin6_port = htons(1212);
8 if(inet_pton(AF_INET6, ADDR, &adrsock.sin6_addr) <= 0)
9     exit(1);
10
11 int r = connect(sock, (struct sockaddr *) &adrsock, sizeof(adrsock));
```

- (a) Quel type de connexion doit accepter le serveur afin qu'une application exécutant ce code puisse s'y connecter ?
- (b) Quel problème de fiabilité pose ce code ? Pourquoi et comment le corriger ?
2. On suppose que deux applications communiquent en mode TCP et que l'application 1 exécute le 1^{er} bout de code en parallèle de l'application 2 qui exécute le 2^e bout de code ci-dessous. En supposant que les appels systèmes ne fassent pas d'erreur, l'application 2 affiche-t-elle quelque chose ? Quand ? Quoi ? Comment corriger ce problème ?

```
1 //application 1
2 char bufsend[SIZE_MESS];
3
4 sprintf(bufsend, "Bonjour le cours de programmation reseaux\n");
5 int ecrit = send(sock1, bufsend, strlen(bufsend), 0);
6 sprintf(bufsend, "Au revoir\n");
7 ecrit = send(sock1, bufsend, strlen(bufsend), 0);
```

```
1 //application 2
2 char bufrecv[SIZE_MESS+1];
3 memset(bufrecv, 0, SIZE_MESS+1);
4
5 int recu = recv(sock2, bufrecv, SIZE_MESS, 0);
6 printf("%s\n", bufrecv);
7 recu = recv(sock2, bufrecv, SIZE_MESS, 0);
8 printf("%s\n", bufrecv);
```

3. Soit le code suivant :

```
1 struct addrinfo hints , *r , *p;
2
3 memset(&hints , 0 , sizeof(hints));
4 hints.ai_family = AF_INET6;
5 hints.ai_socktype = SOCK_STREAM;
6
7 if ((getaddrinfo(hostname , port , &hints , &r)) != 0 || r == NULL)
8     exit(1);
9
10 *addrlen = sizeof(struct sockaddr_in6);
11 p = r;
12 while( p != NULL ){
13     *sock = socket(p->ai_family , p->ai_socktype , p->ai_protocol)
14     if(*sock > 0){
15         if(connect(*sock , p->ai_addr , *addrlen) == 0)
16             break;
17
18         close(*sock);
19     }
20     p = p->ai_next;
21 }
22 printf("%d\n" , ntohs(p->ai_addr->sin6_port));
```

- (a) En supposant que les appels systèmes s'exécutent sans erreur, expliquez ce que fait ce code en précisant le type et les valeurs possibles des variables `hostname` et `port`.
- (b) Expliquez pourquoi la ligne 16 est nécessaire.
- (c) Que faut-il changer dans ce code pour qu'il permette des connexions IPv4 ou IPv6 à un serveur ?

4. Soit le code suivant :

```
1 while(1){
2     int sock2 = accept(sock , NULL , NULL);
3     if(sock2 == -1) {
4         exit(1);
5     }
6 }
```

- (a) Que représentent les variables `sock` et `sock2` ?
- (b) Décrivez les étapes nécessaires à faire avant l'exécution de ce code en précisant les noms des fonctions appelées. Il n'est pas demandé d'écrire du code.
- (c) Complétez ce code afin qu'il permette d'accepter plusieurs clients en parallèle en

utilisant des processus (non légers) puis d'envoyer à chacun le message « OK ». Les processus zombies doivent être récupérés.

5. Donnez les différences entre processus et processus légers.